IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 30 December 2024, accepted 10 January 2025, date of publication 15 January 2025, date of current version 22 January 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3529943

RESEARCH ARTICLE

Bridging Precision and Complexity: A Novel Machine Learning Approach for Ambiguity Detection in Software Requirements

RAHAT IZHAR[1], SHAHID N. BHATTI[2], AND SULTAN A. ALHARTHI[2]
[1]Faculty of Engineering, Chiang Mai University (Presidential Scholarship), Chiang Mai 50200, Thailand
[2]Department of Software Engineering, College of Computer Science and Engineering, University of Jeddah, Jeddah 23890, Saudi Arabia

Corresponding author: Rahat Izhar (rahat_izhar@cmu.ac.th)

This work was supported by the University of Jeddah.

ABSTRACT Ambiguity in software requirements is a significant challenge as it often leads to misunderstandings, implementation errors, and costly project delays. This research proposes a hybrid framework that combines rule-based techniques with machine learning to identify ambiguity in software requirements with precision and efficiency. The framework begins with a rule-based model that systematically detects ambiguities using a carefully prepared list of ambiguous phrases. The analysis utilizes a dataset of 1,553 software requirements drawn from diverse project domains. To capture more intricate ambiguities that traditional rule-based systems might miss, the framework integrates TF-IDF vectorization and a Random Forest classifier, enhancing the precision and coverage of classification. In addition, clustering analysis identifies patterns to provide deeper insights into ambiguous requirements, while sentiment analysis explores the relationship between ambiguity and the emotional tone of requirements. Together, these analyses offer a broader understanding of ambiguity trends and stakeholder perceptions. The framework's performance is validated using standard evaluation metrics, achieving an accuracy of 97%, precision of 97%, recall of 89%, and an F1-score of 92%, significantly surpassing traditional rule-based methodologies. This research advances automatic ambiguity detection by delivering a flexible and interpretable solution. The proposed approach enhances the clarity and quality of software requirements, strengthening requirements engineering practices and supporting more effective software development.

INDEX TERMS Software requirements engineering, natural language processing (NLP), machine learning (ML), requirements ambiguity, artificial intelligence (AI), sentiment analysis.

I. INTRODUCTION

Ambiguity in software requirements is a critical challenge in software engineering, with significant implications for project success [1]. Vague or unclear language in requirements documents often leads to misinterpretations, implementation errors, and project delays, ultimately increasing development costs. As software systems grow more complex and distributed, ambiguity in requirements becomes even more problematic, leading to miscommunication among stakeholders and introducing risks in software development [2]. Despite its importance, addressing ambiguity effectively remains challenging, as existing methods are often insufficient in terms of scalability, accuracy, and interpretability. This research aims to fill this gap by introducing a novel hybrid framework that integrates rule-based techniques with machine learning for ambiguity detection in software requirements.

Traditional ambiguity detection techniques, such as manual reviews and structured walkthroughs, rely heavily on the expertise and intuition of stakeholders [3]. While effective for small projects, these methods are resource-intensive, inconsistent, and unable to scale for large datasets. Automated solutions, including rule-based, natural language processing (NLP) and machine learning (ML) approaches, have emerged to address these limitations. Rule-based methods

The associate editor coordinating the review of this manuscript and approving it for publication was Ayman El-Baz.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

leverage predefined patterns and ambiguous phrases (e.g., ''may,'' ''approximately,'' ''typically''), offering high interpretability but limited flexibility [5], [6], [16]. Conversely, machine learning methods excel in identifying complex patterns and contextual subtleties but often lack the transparency and scalability necessary for industrial application [8]. Neither approach alone is sufficient to handle the diverse and nuanced forms of ambiguity present in real-world requirements [46].

To address these challenges, this study proposes a hybrid framework that combines the strengths of rule-based methods and machine learning models [24], creating a scalable and interpretable solution for ambiguity detection. The framework begins with a rule-based model to detect common ambiguous terms based on a curated list of patterns identified by domain experts. To capture deeper contextual ambiguities, the framework incorporates machine learning using TF-IDF vectorization and a Random Forest classifier. Together, these components address the limitations of standalone techniques, ensuring both precision and broader coverage of ambiguity types. Additionally, clustering analysis is employed to group requirements based on linguistic complexity, while sentiment analysis explores the potential relationship between ambiguity and stakeholder perceptions.

The proposed hybrid framework is validated on a synthesized dataset of 1,553 requirements spanning 17 diverse project domains, including healthcare, e-commerce, finance, logistics, and retail [41], [42]. This dataset is carefully constructed using real-world requirements from publicly available sources, supplemented by expert annotations to identify ambiguous and non-ambiguous requirements. The framework achieves 97% accuracy, 97% precision, 89% recall, and 92% F1-score, outperforming traditional methods and existing state-of-the-art techniques. Beyond its technical contributions, the study offers practical insights into the nature of ambiguous requirements and their potential impact on software engineering practices. The contributions of this research are threefold:

A novel hybrid framework that integrates rule-based methods with machine learning to enhance accuracy, interpretability, and scalability.

A comprehensive dataset, annotated by domain experts, representing ambiguity across a diverse set of software projects.

Advanced techniques, including clustering and sentiment analysis, to provide deeper insights into ambiguity trends and stakeholder perceptions.

This research builds upon existing work in ambiguity detection, such as rule-based methods (e.g., QuARS, RETA) and ML-based models, while addressing their limitations [5], [6], [12]. By offering a systematic, interpretable, and scalable solution, this framework contributes to advancing the state of the art in software requirements engineering [23], with practical implications for reducing miscommunication and improving software development project outcomes and software quality [45].

The remainder of this paper is structured as follows: Section II provides a detailed overview of the related work, highlighting advancements and gaps in ambiguity detection techniques. Section III describes the methodological approach, including dataset characteristics, preprocessing steps, and the hybrid framework combining rule-based and machine learning methods. Section IV presents the evaluation metrics and experimental results, along with a comparative analysis of different machine learning models and rule-based tools. Section V discusses the findings, practical implications, limitations, and potential avenues for future research. Finally, Section VI concludes the study by summarizing the contributions and emphasizing the significance of the proposed framework for improving software requirements engineering practices.

II. SCHOLARLY SYNTHESIS

Addressing ambiguity in software requirements is crucial for minimizing project risks and improving communication among stakeholders. Various approaches, including rule-based techniques and machine learning models [25], have been proposed to detect and mitigate ambiguity. These methods aim to improve the clarity of requirements by identifying vague or imprecise language that can lead to misinterpretations. The following studies explore key advancements and challenges in this domain, highlighting the evolution of ambiguity detection techniques.

Kato et al. [1] provide a novel method to mitigate ambiguity in software requirements specifications via the creation of a knowledge dictionary centered on transitive verbs and their objects. This dictionary seeks to discover ambiguities by recognizing imprecise or ambiguous language in requirement papers. The approach, grounded on the procurement requirements of Japan's Ministry of Health, Labor, and Welfare, utilizes text mining and pattern matching methods to identify possible ambiguities in draft procurement specifications. The research recognizes limits in accuracy stemming from the intricacies of precisely recording every ambiguous phrase, while emphasizing good memory to prevent the omission of ambiguous terms. Despite its shortcomings, the research significantly contributes to early-stage requirement analysis by using natural language processing (NLP) to automatically identify ambiguities, therefore reducing the likelihood of project delays.

Berry and Kamsties [2] established a thorough framework for comprehending and tackling ambiguity, categorizing it into four principal types: lexical, syntactic, semantic, and pragmatic. Their manual methodology, including taxonomies and checklists to assist engineers in recognizing ambiguous statements, is pragmatic but constrained in scalability, particularly for extensive projects. They emphasized the urgent need for more systematic detection approaches, since dependence on human skill often resulted in uneven outcomes. Although their paradigm was impactful, they underscored the need of semi-automated and automated methods to effectively address uncertainty in complex software systems.

De Bruijn and Dekkers [3] examined the influence of unclear natural language requirements on projects success, centering on a case study of a failed intricate data processing system. The authors challenged the conventional perspective that ambiguity is a primary contributor to project failure, proposing other elements such as communication and project complexity may be more significant determinants. Their research indicated that although ambiguity existed in a considerable proportion of requirements, it accounted for only a minor percentage of project failures. This observation challenges the prevalent belief in software engineering, highlighting the need for a more nuanced understanding of its impact.

Bajceta et al. [5] examined the efficacy of four natural language processing (NLP) tools—Automated Requirements Measurement (ARM), Quality Analyzer for Requirement Specifications (QuARS), REquirements Template Analyzer (RETA), and Requirements Complexity Measurement (RCM)—in identifying ambiguities within system requirements. The research used a dataset of 180 requirements from the industrial partner Alstom to compare the results of the tools with expert evaluations. Although the tools often attained good recall, accuracy was comparatively poor, resulting in several false positives. RETA had the greatest recall, recognizing virtually all ambiguous criteria, but RCM attained the maximum accuracy. These results underscore the difficulty of reconciling accuracy and recollection in automated ambiguity identification and indicate the need for more advanced techniques to enhance precision without compromising recall.

Fantechi et al. [6] conducted a comparative research of rule-based NLP tools and ChatGPT for the identification of vaugguity in requirements documents. Although conventional methods like QuARS demonstrated superior accuracy, their findings indicated that ChatGPT identified certain fault categories, such as incompleteness, that QuARS overlooked. Notwithstanding ChatGPT's inconsistent effectiveness in various instances, the research indicates its potential as an auxiliary tool. Nonetheless, challenges related to consistency and explainability persist as constraints requiring more exploration.

Ferrari et al. [7] proposed a way for identifying pragmatic ambiguity in natural language needs using graph-based modeling. Their approach employs a shortest-path search algorithm and domain knowledge graphs derived from pertinent documents to replicate the real-world comprehension of requirements, hence identifying ambiguity. Their technique, tested on a publicly accessible paper, demonstrated an accuracy of 51% and a recall of 63%, highlighting the need for additional enhancements to increase its efficacy, especially in addressing pragmatic ambiguities.

Alharbi [8] investigated ambiguity detection with a refined transformation method. They contrasted two methodologies: feature extraction using hidden states to train a Support Vector Machine (SVM) and an end-to-end model refined on a pre-trained DistilBERT model. The meticulously calibrated model significantly enhanced ambiguity detection, with an F1-score of 77%. This underscores the efficacy of transformer models in this field, illustrating their superiority compared to conventional classifiers such as Logistic Regression, SVM, and KNN.

Satpute and Agrawal [9] investigated the identification of pragmatic ambiguities in natural language requirements (NLRs) using a multi-step methodology including contextual analysis and rephrasing of ambiguities. Field experts affirmed that their methodology significantly enhanced ambiguity detection. The Random Forest classifier had an accuracy of 86.66%, surpassing other models such as SVM and K-NN. The research highlights the significance of specialized knowledge in clarifying ambiguity and indicates that progress in AI and machine learning may further improve detection capacities.

Shah and Jinwala [10] performed an extensive examination of current tools and methodologies for addressing ambiguity in software specifications. They emphasized tools like SREE and RESI for their superior recall, despite these instruments still facing challenges with ambiguous vagueness and intricate coordination. The research revealed that while existing technologies have potential, they cannot entirely eradicate ambiguities without human involvement owing to the intricacies of spoken language. This underscores the persistent need for sophisticated tools that integrate human and machine skills.

Kamsties and Peach [11] introduced a checklist-based approach for identifying ambiguity in natural language requirements, distinguishing between linguistic ambiguities and those related to requirements engineering. Their technique, while successful, is significantly dependent on the inspectors' competence, resulting in less scalability and consistency. This reliance on manual inspections restricts its application to extensive requirements documents. Matsuoka and Lepage [12] used WordNet semantic similarity and the C-value metric to detect ambiguous phrases in Software Requirements Specifications (SRS). Their methodology demonstrated the efficacy of semantic similarity in identifying confusing terminology, especially in technical circumstances. Although their approach had potential in enhancing the clarity of SRS papers, it encountered difficulties in generalizability across many domains.

The reviewed literature highlights persistent challenges in ambiguity detection within software requirements, particularly in balancing precision, scalability, and interpretability. Rule-based methods demonstrate effectiveness in identifying explicit ambiguities through predefined patterns but lack the flexibility to address context-dependent and nuanced cases [21]. Machine learning approaches, while capable of detecting intricate patterns, often suffer from issues of explainability and reliance on large, annotated datasets [14], [27]. These findings emphasize the critical need for integrated methodologies that leverage the interpretability of

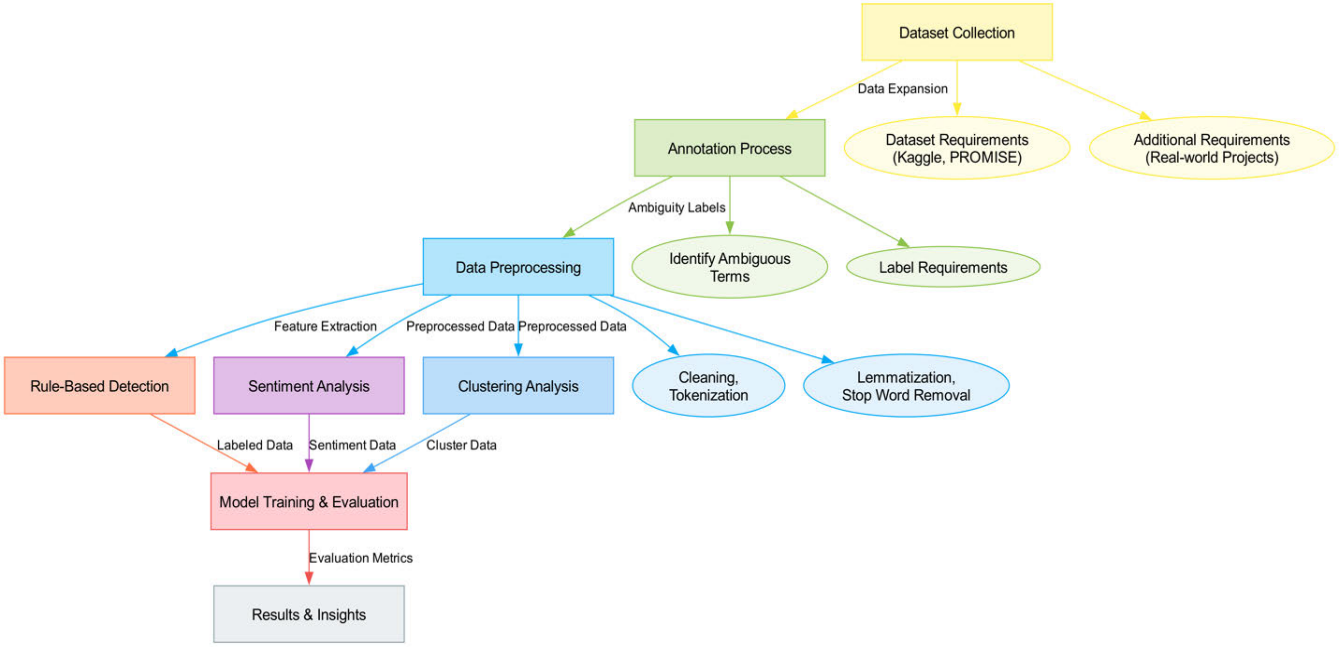


FIGURE 1. Methodology workflow roadmap.

rule-based systems alongside the adaptability of machine learning techniques. Such hybrid approaches are positioned as a promising avenue for advancing the accuracy and reliability of ambiguity detection, addressing the complexities inherent in software requirements engineering and development [47].

III. METHODOLOGICAL APPROACH

The following proposed approach combines rule-based techniques with machine learning to detect ambiguity in software requirements. This dual methodology not only improves detection accuracy but also enhances the interpretability of identified ambiguities—an aspect often lacking in machine learning-based models. Additionally, the integration of sentiment analysis provides valuable insights into stakeholder perceptions, a perspective that is rarely explored in existing research. This study follows a structured methodology designed to systematically detect ambiguity in software requirements through a combination of rule-based and machine learning techniques. The research process is divided into several key stages, each addressing a specific aspect of ambiguity detection, from data collection and preprocessing to model training and evaluation [20]. The experiments were conducted on a MacBook Pro running macOS Sequoia (Version 15.1.1), equipped with a 2.3 GHz Quad-Core Intel Core i7 processor, 16 GB 1600 MHz DDR3 RAM, and Intel Iris Pro Graphics (1536 MB). The software environment included Python 3.9.13 with libraries such as Scikit-learn 1.4.0, NumPy 1.24.4, Pandas 1.4.4, TensorFlow 2.16.2, Transformers 4.36.2, TextBlob 0.17.1, and NLTK 3.7. Figure 1 presents a visual roadmap of the methodology, offering a comprehensive overview of the steps involved in this study.

A. DATASET COLLECTION AND EXPANSION

The dataset used in this study was initially obtained from the publicly available Software Requirements Dataset repository on Kaggle and the PROMISE dataset on Zenodo, which collectively contained 976 software requirements from various projects [41], [42]. To create a more comprehensive and representative dataset, additional requirements were collected from multiple projects documentations across diverse industries, including manufacturing, supply chain, e-commerce, healthcare, finance, logistics, and retail. This extended the dataset to a total of 1,553 unique software requirements, encompassing 17 distinct project domains.

The additional requirements were sourced from real-world projects undertaken by industrial software houses, and setups. These were extracted from project documentation and publicly available resources, ensuring a diverse representation of software requirements typically encountered in various industrial settings. The expanded dataset aimed to capture the complexity and diversity of software requirements, providing a robust foundation for training and evaluating the ambiguity detection model. Table 1 shows the data characteristics. By emphasizing the dataset's diversity across 17 domains, the inclusion of real-world requirements from varied industries, and its adequacy for machine learning tasks, the response demonstrates that the dataset is both comprehensive and reflective of real-world challenges in requirements engineering. This directly supports the framework's applicability and credibility in practical software development [49] scenarios.

Each requirement was then reviewed and annotated by domain experts with extensive experience in software requirements engineering. They identified ambiguous terms and phrases commonly found in software requirements, such as ''may'', ''could'', ''typically'', and ''as needed'', which

TABLE 1. Overview of dataset characteristics.

Parameter	Value
Total Requirements	1,553
Original Requirements (PROMISE)	976
Additional Requirements	577
Number of Project Domains	17
Industries Covered	Manufacturing, Supply Chain, E-Commerce, Healthcare, Finance, Logistics

could lead to multiple interpretations. Table 2 presents a sample of ambiguous terms and their explanations. A comprehensive list of such terms was compiled to serve as the basis for labeling the dataset. Clear annotation guidelines were established to ensure consistency across annotations. Each requirement was labeled as either ''Ambiguous'' or ''Not Ambiguous'' based on these guidelines, and the annotations were cross validated by five experts to minimize bias and ensure high inter-annotator agreement. This step was crucial for achieving a reliable labeled dataset that could be used to train and validate the ambiguity detection models.

TABLE 2. Sample ambiguous terms identified by domain experts.

Ambiguous Term	Example Usage	Explanation
may	"The system may provide additional support."	Indicates a potential feature that is not clearly defined.
could	"The product could potentially be used for..."	Suggests possibilities without commitment or clear criteria.
generally	"This feature generally enhances performance."	Vague quantifier that lacks specificity about conditions or extent.
approximately	"The system shall refresh approximately every 5 minutes."	Imprecise time frame that could lead to varying interpretations.

The entries in the table 3 illustrate examples of both non-ambiguous and ambiguous software requirements, emphasizing the importance of linguistic precision in requirement documentation.

Non-ambiguous examples include requirements that are clear, actionable, and measurable, leaving no room for misinterpretation. For instance, *Requirement 1* specifies a precise interval (''every 60 seconds'') and a clear action (''refresh the display''), ensuring that all stakeholders interpret it in the same way. Similarly, *Requirement 3* provides measurable criteria, such as ''10×10 projection screen,'' ''90% of viewers,'' and ''distance of 30,'' which establish explicit expectations and boundaries, making the requirement unambiguous and easy to implement.

On the other hand, **ambiguous examples** highlight requirements that use subjective or vague language, leading

TABLE 3. Ambiguous and non-ambiguous requirements examples from dataset.

Requirement Text	Ambiguity Label	Ambiguity Type
1. The system shall refresh the display every 60 seconds.	Not Ambiguous	-
7. The product shall be intuitive and self-explanatory. 90% of new users shall be able to start the display of Events or Activities within 90 minutes of using the product.	Ambiguous	Vague Terms
3. If projected, the data must be readable on a 10x10 projection screen; 90% of viewers must be able to read Event / Activity data from a viewing distance of 30.	Not Ambiguous	-
26. The product shall be easy for a realtor to learn.	Ambiguous	Subjective Language

to potential misunderstandings. For example, *Requirement 7* employs terms like ''intuitive'' and ''self-explanatory,'' which lack concrete definitions and are open to interpretation. While it includes a quantifiable condition (''90% of new users''), the qualitative aspect introduces ambiguity. Similarly, *Requirement 26* uses subjective language, such as ''easy for a realtor to learn,'' without providing specific criteria or measurable outcomes, leaving room for varying interpretations of what constitutes ''ease of learning.'' These examples underscore the necessity of eliminating ambiguity in software requirements to ensure clarity and alignment among stakeholders.

B. DATA PREPROCESSING AND FEATURE ENGINEERING

Preprocessing is a critical step that transforms raw text data into a structured format suitable for analysis [28]. The following steps were implemented to clean and prepare the dataset for machine learning and statistical analysis.

1) PHASE 3. BASIC TEXT CLEANING

The initial phase of preprocessing involved basic text cleaning to standardize the requirements and remove noise [12], [13]. This process included the removal of special characters, such as punctuation marks, to eliminate irrelevant elements that do not contribute to the meaning of the text. Additionally, the text was converted to lowercase to ensure consistency and prevent the model from treating words like ''System'' and ''system'' as different entities. Finally, leading, trailing, and multiple consecutive spaces were trimmed to avoid unnecessary fragmentation of the text.

2) ADVANCED PREPROCESSING

Advanced text preprocessing techniques were applied to further refine the dataset. This included tokenization, where the text was split into individual tokens (words) to enable detailed analysis at the word level. For example, the requirement ''The system shall provide real-time updates'' was tokenized

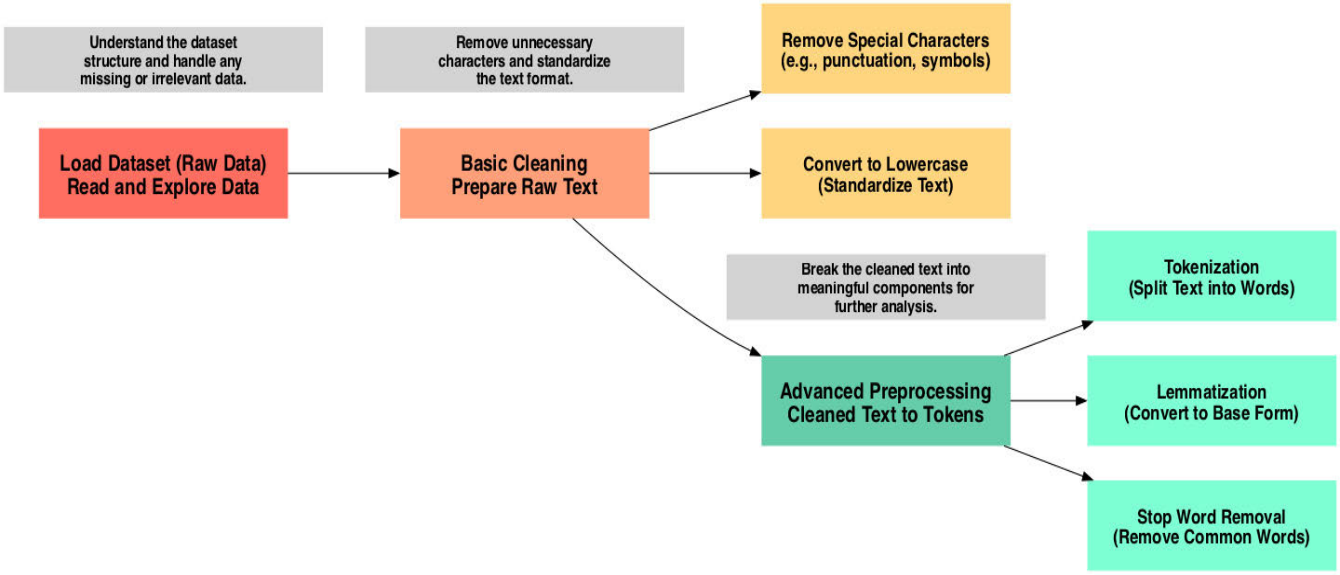


FIGURE 2. Feature extraction and vectorization workflow.

into [''the'', ''system'', ''shall'', ''provide'', ''real-time'', ''updates'']. Lemmatization was then applied to reduce words to their base form, consolidating different inflections of a word into a single representative form. For instance, the words ''running'' and ''ran'' were both lemmatized to ''run.'' Additionally, stop word removal was performed to eliminate common English stop words (e.g., ''and'', ''the''), allowing the focus to remain on the more meaningful content of the requirements. As a result, the requirement ''The system is user-friendly'' was transformed into ''system user-friendly'' for analysis [18], [19]. The data preprocessing workflow is illustrated in Figure 2, which outlines the sequential steps involved in transforming raw text data into a format suitable for further analysis. The workflow includes stages such as basic cleaning, tokenization, lemmatization, and stop word removal.

A summary of the text preprocessing techniques applied in this study, along with example transformations, is provided in table 4. This table clearly demonstrates how each preprocessing step modifies the text data, ensuring consistency and enhancing the quality of the features used for subsequent analysis.

3) FEATURE EXTRACTION

Several features were extracted from the preprocessed text to support further analysis and model training. These features include word count, which represents the total number of words in each requirement, providing insights into the verbosity of the text. For example, the requirement ''The system shall provide real-time updates'' has a word count of 6. Sentence length, measured as the total number of characters in each requirement, was also extracted to indicate the complexity of the text, with the same example having

TABLE 4. Summary of text preprocessing techniques.

Technique	Description	Example Transformation
Basic Cleaning	Remove special characters, convert to lowercase	"System updates!!!" → "system updates"
Tokenization	Split text into individual words	"system updates" → ["system", "updates"]
Lemmatization	Convert words to their base form	["updates", "running"] → ["update", "run"]
Stop Word Removal	Remove common words that do not add significant meaning	["the", "system", "is"] → ["system"]

a sentence length of 39 characters [31]. Additionally, the complex word percentage was calculated by determining the proportion of words in each requirement that have more than six characters, serving as a measure of linguistic complexity. In the example ''The system shall provide real-time updates,'' 2 out of 6 words (''provide,'' ''updates'') are complex, yielding a complex word percentage of 33%. Finally, the average word length, defined as the mean number of characters per word in each requirement, was used to gain further insight into the readability of the text. For instance, the average word length in the requirement ''The system shall provide real-time updates'' is 5.5 characters.

This comprehensive preprocessing and feature extraction approach ensured that the dataset was well-prepared for the subsequent rule-based and machine learning models. A detailed overview of these features, along with specific examples, is presented in table 5. This table illustrates the transformations from raw text to preprocessed data and the calculated features, including word count, sentence length,

TABLE 5. Example preprocessing and features extracted.

Requirements	Cleaned Requirements	Preprocessed Requirements	Word Count	Sentence Length	Complex Word Percentage	Average Word Length
The system shall refresh the display every 60 seconds.	the system shall refresh the display every 60 seconds	system shall refresh display every 60 second	7	44	28.57%	5.43
The application shall match the color of the screen.	the application shall match the color of the screen	application shall match color screen	5	36	20.00%	6.40
If projected, the data must be readable on a 100-inch screen.	if projected the data must be readable on a 100 inch screen	projected data must readable 100 inch screen	7	44	28.57%	5.43

complex word percentage, and average word length. In this initial phase, basic features such as word count, sentence length, complex word percentage, and average word length were extracted for the entire dataset. These features provided a foundational understanding of the dataset and were utilized in the initial machine learning models to establish a baseline performance.

C. RULE-BASED AMBIGUITY DETECTION

A rule-based model was developed to automatically detect ambiguity in software requirements. This model leveraged the list of ambiguous terms identified by domain experts to label each requirement as ''Ambiguous'' or ''Not Ambiguous.'' The rule-based model employed pattern matching techniques using regular expressions to detect the presence of ambiguous terms in the text. The model was constructed using a set of predefined rules based on the ambiguous terms and patterns identified by experts. These rules were applied to the preprocessed text data to automatically label requirements. If a requirement contained one or more ambiguous terms, it was labeled as ''Ambiguous''; otherwise, it was labeled as ''Not Ambiguous.'' The rule-based approach served as an initial filter to quickly identify and categorize requirements based on predefined patterns of ambiguity. However, recognizing that not all forms of ambiguity could be captured through predefined rules, further analysis was conducted to enhance the understanding of the dataset.

The figure 3 illustrates the workflow of the rule-based ambiguity detection process. It starts with input requirements undergoing initial cleaning, followed by advanced preprocessing that includes tokenization, lemmatization, and stop word removal. The preprocessed data is then matched against a pattern library of ambiguous terms. Requirements that match these patterns are labeled as ''Ambiguous'', while those that do not match are labeled as ''Not Ambiguous''. The final labeled output is used for further analysis and training.

The bar chart figure 4 illustrates the distribution of software requirements based on their ambiguity labels. It shows a comparison between the number of requirements classified as ''Not Ambiguous'' and those labeled as ''Ambiguous.'' The chart indicates that out of the total requirements analyzed, a significant majority, 1371 requirements, are considered ''Not Ambiguous,'' while only 185 requirements are labeled as ''Ambiguous.'' This advocates that most requirements in the dataset are clear and unlikely to cause misunderstandings, whereas a smaller portion contains language that could be interpreted in multiple ways, potentially leading to confusion during implementation or communication. The numerical values displayed above the bars provide a clear count for each category, emphasizing the disparity between ambiguous and non-ambiguous requirements in the dataset.

D. SENTIMENT ANALYSIS

To complement the rule-based ambiguity detection, sentiment analysis was conducted to evaluate the emotional tone of the requirements. Understanding sentiment provides valuable insights into stakeholder expectations and potential conflicts within the requirements [38]. The TextBlob library was utilized to assign a polarity score to each requirement. A positive sentiment indicates a favorable or optimistic tone, suggesting that stakeholders are satisfied or have high expectations [39]. In contrast, a negative sentiment reflects concerns or dissatisfaction, highlighting potential areas of conflict or issues that need to be addressed [40]. A neutral sentiment suggests that no significant emotional tone is expressed in the requirement, indicating objectivity.

The sentiment of each requirement was classified based on its polarity score, as summarized in table 6. Requirements with a polarity score greater than zero were categorized as having a positive sentiment, indicating a generally favorable or optimistic perception. A polarity score less than zero was categorized as having a negative sentiment, reflecting concerns or dissatisfaction. A score equal to zero indicated a neutral sentiment, suggesting that the requirement was stated objectively without any particular emotional tone.

TABLE 6. Sentiment classification criteria.

Sentiment	Polarity Range
Positive	> 0
Negative	< 0
Neutral	$= 0$

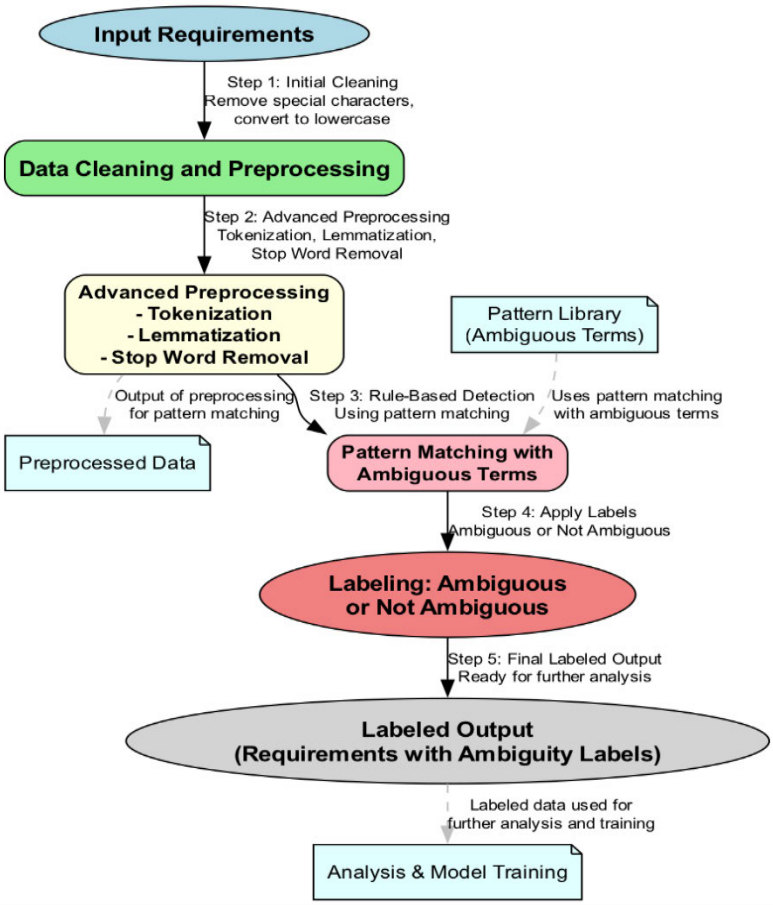


FIGURE 3. Flow of rule-based ambiguity detection model.

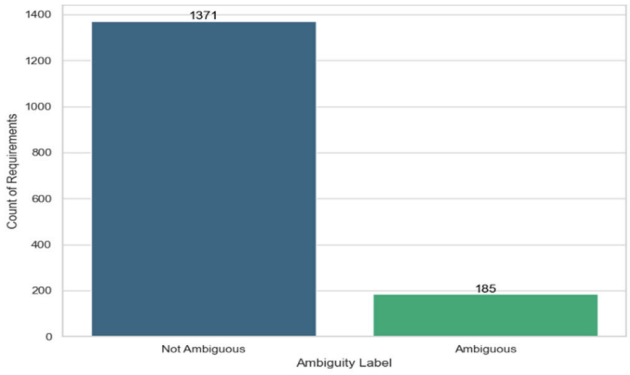


FIGURE 4. Distribution of ambiguity labels.

1) CORRELATION ANALYSIS BETWEEN AMBIGUITY AND SENTIMENT

To explore the relationship between ambiguity and sentiment in software requirements, a correlation analysis was conducted [43]. Ambiguity refers to unclear or vague language in requirements [15], which can lead to multiple interpretations, while sentiment reflects the emotional tone of the requirements, indicating whether they are perceived as positive, negative, or neutral [38]. Both the ambiguity and sentiment labels were converted into numerical values for analysis, and the Pearson correlation coefficient was calculated. The analysis revealed a weak positive correlation ($r = 0.22$), suggesting that as the level of ambiguity in the requirements slightly increases, the sentiment associated with them tends to become more positive [34].

Although this relationship is not very strong, it indicates that some stakeholders may perceive ambiguous requirements in a more favorable or optimistic light. This could be because ambiguity allows for flexibility or room for interpretation, which might be seen as a positive aspect in certain contexts. However, this trend is not consistent and should be interpreted with caution, as ambiguity can still lead to misunderstandings and conflicts in other scenarios. This finding highlights the complex nature of stakeholder perceptions regarding ambiguous requirements and suggests that while some ambiguity might be tolerated or even appreciated, clarity remains crucial for effective communication and project success.

Figure 5 illustrates the sentiment distribution of software requirements, categorizing them into three groups: Neutral, Positive, and Negative. The majority of requirements (803)

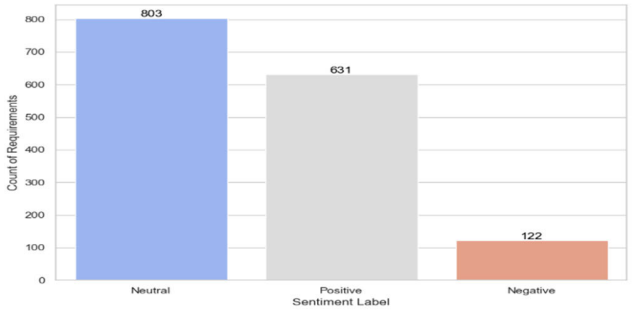


FIGURE 5. Distribution of sentiment labels.

fall under the Neutral category, indicating that they are expressed in a straightforward and objective manner without conveying any strong emotional tone. This neutrality suggests that most requirements are focused on technical or functional aspects, minimizing subjective interpretations.

The Positive category, comprising 631 requirements, reflects language that conveys optimism or favorable perceptions, such as well-defined functionalities or features that align with stakeholder expectations. For example, a requirement like: ''The system shall provide real-time updates,'' demonstrates clarity and specificity, which can foster confidence and satisfaction among stakeholders.

The Negative sentiment category, the smallest at 122 requirements, represents concerns or dissatisfaction, often stemming from vague or incomplete language [17]. For instance, a requirement like: ''The product could potentially support the following functions,'' introduces uncertainty with terms like ''could'' and ''potentially,'' which may lead to misaligned stakeholder expectations or potential conflicts.

Overall, the sentiment distribution not only highlights the predominance of neutral requirements but also reveals that requirements with Positive sentiment are often clearer and more precise, while those with Negative sentiment frequently overlap with ambiguous language. These findings provide actionable insights for refining requirements documentation to reduce ambiguity and improve stakeholder alignment.

E. CLUSTERING ANALYSIS

Clustering analysis was performed to identify natural groupings within the dataset based on extracted textual features such as word count, sentence length, complex word percentage, and average word length [37]. This approach helps uncover hidden patterns among the requirements, providing deeper insights into their characteristics and guiding model training and evaluation. Importantly, the clustering results can inform how sentiment and ambiguity manifest across different requirement types.

1) FEATURE SELECTION FOR CLUSTERING

The features selected for clustering capture key textual properties of the requirements:

a: WORD COUNT

Indicates the verbosity and level of detail in a requirement.

b: SENTENCE LENGTH

Reflects the complexity and elaboration of the text.

c: COMPLEX WORD PERCENTAGE

Measures the proportion of complex terms, showing the linguistic and technical complexity of the requirement.

d: AVERAGE WORD LENGTH

Provides insight into the readability and sophistication of the language used. These features were chosen because they effectively differentiate between types of requirements, enabling the clustering algorithm to group similar requirements together based on their textual characteristics

2) K-MEANS CLUSTERING

The K-means algorithm was employed to group the requirements into three distinct clusters based on their feature similarities as shown in FIGURE 6. This algorithm iteratively adjusts the positions of cluster centers to minimize variance within each cluster and maximize the distinction between clusters [35], [36]. The final clusters reveal underlying patterns in the requirements, facilitating a more nuanced understanding of their characteristics.



FIGURE 6. Clustering analysis of requirements.

Figure 6 illustrates the K-means clustering results, where software requirements are grouped into three clusters based on sentence length (x-axis) and complex word percentage (y-axis).

Cluster 0 (red circles): Represents longer and more detailed requirements, which may increase the risk of ambiguity.

Cluster 1 (blue squares): Contains shorter but more linguistically complex requirements.

Cluster 2 (green diamonds): Includes moderately long requirements with balanced complexity, making them clearer and easier to interpret.

The clustering analysis revealed distinct patterns in ambiguous requirements, such as longer, more complex sentences being more prone to ambiguity (Cluster 1). These insights allow targeted refinement of high-risk requirements, potentially reducing ambiguity before they are fed into the machine learning model.

3) RATIONALE FOR CHOOSING THREE CLUSTERS

Three clusters were selected for the following key reasons:

a: ELBOW METHOD

The elbow method was employed to evaluate the within-cluster sum of squares (WCSS) for various numbers of clusters. As observed in the plotted graph, the curve exhibits a distinct "elbow" at three clusters. This indicates a point where adding more clusters yields diminishing returns in terms of reduced WCSS [35]. Therefore, three clusters were chosen as the optimal balance between cluster compactness and separation.

b: INTERPRETABILITY

Choosing three clusters provides a clear and manageable segmentation of the dataset. This allows for more straightforward interpretation of the results, making it easier to derive actionable insights without introducing unnecessary complexity that could arise from a larger number of clusters.

c: DATA CHARACTERISTICS

The selected features, such as word count, sentence length, complex word percentage, and average word length, suggest that three clusters effectively capture the primary patterns in the data. This clustering helps distinguish between different types of requirements, such as simpler, more complex, and moderately detailed requirements, providing a meaningful categorization that reflects the inherent structure of the dataset.

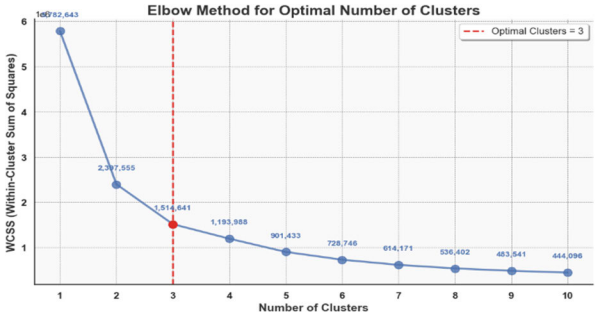


FIGURE 7. Optimal number of clusters determined using the elbow method.

The Elbow Method, illustrated in figure 7, is used to identify the optimal number of clusters by plotting WCSS values for clusters ranging from 1 to 10 [35]. The x-axis represents the number of clusters, while the y-axis displays the corresponding WCSS values. A red dotted line in the figure highlights the point at three clusters, where a significant drop in WCSS occurs. This indicates that increasing the number of clusters beyond three offers minimal improvement to the model, confirming three as the optimal choice for this dataset.

4) CLUSTER CHARACTERISTICS AND INSIGHTS FROM CLUSTERING

The clustering analysis identified three distinct groups of requirements, each characterized by unique textual properties:

Cluster 0: Comprised of requirements that are concise and straightforward. These requirements feature shorter sentences and lower word counts, making them clear and less prone to ambiguity. This simplicity suggests that the requirements in this cluster are likely to be well-defined and easier to implement.

Cluster 1: Contains more complex and detailed requirements. These requirements are marked by higher word counts and a greater proportion of complex words. Such complexity indicates the presence of technical details that may require additional clarification to ensure comprehensive understanding among stakeholders. This cluster may be more challenging to interpret and could benefit from further refinement.

Cluster 2: Represents requirements with moderate verbosity but higher average word length. Despite the use of sophisticated language, these requirements remain relatively clear. The balanced nature of these requirements indicates a good equilibrium between detail and comprehensibility, making them well-suited for effective communication.

TABLE 7. Cluster characteristics.

Cluster	Word Count	Sentence Length	Complex Word Percentage	Average Word Length
0	39.67	254.69	54.97	6.80
1	10.98	73.17	52.65	6.95
2	21.53	136.43	48.25	6.65

Table 7 summarizes the average characteristics of each cluster, highlighting the distinctions in word count, sentence length, complex word percentage, and average word length. For clustering analysis, a refined set of features was extracted, focusing specifically on identifying natural groupings within the dataset. While the features, such as word count and sentence length, were similar in nature to those used in the initial phase, their application was tailored to highlight patterns and relationships unique to different clusters. This approach enabled a deeper understanding of distinct requirement groups, revealing insights that were not apparent in the general analysis. The clustering analysis provides valuable guidance for refining requirement documentation. Each cluster exhibits unique characteristics, indicating specific areas that may require attention for improving clarity and effective communication.

Simpler Requirements (Cluster 0): These requirements are likely clear and unambiguous, suggesting that they are well-defined and more straightforward to implement. They serve as a model for drafting clear and effective requirements.

Complex Requirements (Cluster 1): The detailed and technical requirements in Cluster 1 may need further review to ensure they are well-understood by all stakeholders. Efforts should be made to simplify these requirements or provide additional context to avoid misinterpretation.

Balanced Requirements (Cluster 2): The requirements in this cluster strike a good balance between detail and clarity. They use sophisticated language without sacrificing comprehensibility, making them well-suited for conveying complex ideas in a clear manner.

This analysis highlights areas where improvements in clarity and specificity are needed, providing guidance for refining the documentation of requirements. Such enhancements can significantly improve project communication and understanding, ultimately contributing to project success.

F. CLUSTER-WISE ANALYSIS

After clustering, an analysis was performed to understand the distribution of ambiguity and sentiment labels within each cluster. This step helps in identifying whether certain clusters are more prone to ambiguity or specific sentiments, providing deeper insights into the nature of the requirements.

TABLE 8. Cluster-wise distribution of ambiguity labels.

Cluster	Ambiguous Count	Not Ambiguous Count
0	23	98
1	115	890
2	47	383

Table 8 shows the number of ambiguous and non-ambiguous requirements in each cluster, highlighting which clusters contain a higher proportion of ambiguous requirements.

TABLE 9. Cluster-wise distribution of sentiment labels.

Cluster	Negative Sentiment	Neutral Sentiment	Positive Sentiment
0	18	29	74
1	61	599	345
2	43	175	212

Table 9 provides a breakdown of sentiment labels across different clusters, helping to identify whether certain clusters are associated with particular sentiments. The cluster-wise analysis provided critical insights into the distribution of ambiguity and sentiment across different requirement clusters. Cluster 1, for instance, had the highest proportion of ambiguous requirements and negative sentiment, suggesting a potential area for targeted refinement and clarification. In contrast, Cluster 0 showed a lower incidence of ambiguity and more positive sentiment, indicating clearer and well-understood requirements. These insights not only reveal the varying nature of requirements but also highlight areas that may require additional attention to mitigate ambiguity and improve stakeholder satisfaction.

These insights reveal the varying nature of requirements and highlight areas that may require additional attention to mitigate ambiguity and improve stakeholder satisfaction. This cluster-level understanding is crucial for guiding efforts to enhance the clarity and effectiveness of requirements documentation, ultimately contributing to more successful and efficient project execution.

G. MACHINE LEANING MODEL TRAINING AND FEATURE ENGINEERING

Random Forest classifier was used to effectively identify ambiguous software requirements in this research. This approach leverages the power of TF-IDF to convert textual data into numerical features, while insights gained from prior analyses—specifically, rule-based ambiguity detection, sentiment analysis, and clustering—enhance the robustness of the Random Forests in classifying these features into ambiguous and non-ambiguous categories [33].

TF-IDF vectorization was selected for feature extraction due to its ability to weigh words based on their significance within a document and their rarity across the entire dataset. Unlike the Bag of Words model, which only considers word frequency, TF-IDF [50] assigns higher importance to terms that are distinct and potentially indicative of ambiguity [30]. This is particularly relevant in the context of the findings from the rule-based analysis, which identified key ambiguous terms that TF-IDF can highlight while minimizing the impact of common terms like ''system'' or ''software''

Term Frequency (TF) measures the frequency of a term within a specific document. It is calculated as follows in equation eq (1):

$$TF\,(t,d) = \frac{Number\ of\ total\ occurances\ of term\ t\ in\ document\ d}{Total\ number\ of\ terms\ in\ document\ d} \tag{1}$$

This value indicates how important a term is within a particular document, aiding in the detection of terms related to ambiguity identified in earlier analyses. Inverse Document Frequency (IDF) down-weights terms that appear frequently across many documents, emphasizing terms that are more unique and informative. The IDF value is calculated as in eq (2):

$$IDF\,(t) = log\left(\frac{N}{|\{d \in D : t \in d\}|}\right) \tag{2}$$

where:

- N is the total number of documents in the dataset,
- $|\{d \in D{:}t{\in}d\}|$ is the number of documents contining the term t.

This metric ensures that the model focuses on the more unique features that contribute to ambiguity and sentiment,

which were highlighted in the previous analyses. The TF-IDF score for a term in a document is computed by multiplying its TF and IDF values as in eq (3):

$$TF - IDF\ (t, d) = TF\ (t, d) \times IDF(t) \tag{3}$$

This score quantifies both the importance of a term within a document and its rarity across the dataset. The sklearn library was used to transform the preprocessed requirements into numerical features. The steps included:

1) INITIALIZATION

Configuring the vectorizer with max_features=1000 to focus on the most relevant terms.

2) TRAINING THE VECTORIZER

Applying the fit_transform method to the training data (X_train) to generate a TF-IDF feature matrix.

3) TRANSFORMING THE TEST SET

Using the transform method on the test data (X_test) to ensure consistent feature representation.

This process converts the text data into a sparse matrix, where each row represents a requirement, and each column represents a unique term. The values in the matrix correspond to the TF-IDF scores, which serve as input for the Random Forest classifier The Random Forest classifier was employed due to its robustness in handling high-dimensional data and its ability to capture complex, non-linear relationships between features. It benefits from insights obtained during rule-based and sentiment analysis stages, which contribute to a more nuanced understanding of the data. The key steps in the Random Forest model are as follows:

4) BOOTSTRAP SAMPLING

Each decision tree in the forest is trained on a different subset of the training data, selected through random sampling with replacement [29], [44].

5) RANDOM FEATURE SELECTION

For each split in a tree, a random subset of features is chosen, reducing overfitting and ensuring diversity among the trees.

6) TREE CONSTRUCTION

Each tree is grown to a maximum depth based on the criteria set during initialization.

7) AGGREGATION OF PREDICTIONS

For classification, predictions from all trees are aggregated through majority voting to determine the final class label.

The dataset was split into 80% for training and 20% for testing, ensuring the model can generalize well to unseen data. This approach, enhanced by the insights from previous analyses, provides a robust framework for effective ambiguity detection in software requirements.

H. EVALUATION AND PERFORMANCE ANALYSIS

The hybrid framework was evaluated using standard metrics, including accuracy, precision, recall, and F1-score, which provide a holistic view of the model's effectiveness in distinguishing between ambiguous and non-ambiguous requirements [32].

1) ACCURACY

The model achieved a high accuracy of 97%, indicating that the majority of the predictions made were correct. This high level of accuracy demonstrates the model's ability to generalize well across a diverse dataset containing requirements from various domains. The integration of rule-based methods with machine learning significantly enhances the model's capability to handle complex requirements that traditional methods might overlook. The accuracy metric is essential in understanding the overall performance of the model, as it reflects the proportion of correctly classified requirements out of the total number of requirements in the test set. A high accuracy score is indicative of the model's robustness and its potential to be applied in real-world scenarios where accurate classification of requirements is crucial to avoid misinterpretations and costly project delays. Accuracy is calculated as in eq (4):

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{4}$$

2) PRECISION

Precision, which measures the proportion of true positive predictions out of all positive predictions made by the model, was also found to be 97%. This high precision score indicates that when the model identifies a requirement as ambiguous, it is highly likely to be correct. This is particularly important in requirements engineering because a high precision minimizes false positives, thereby reducing the risk of unnecessary rework and resource allocation. False positives in requirements analysis can lead to confusion, additional clarification cycles, and wasted efforts in addressing nonexistent ambiguities. Therefore, the high precision achieved by the model suggests that it can effectively filter out non-ambiguous requirements, allowing stakeholders to focus on the truly ambiguous ones. Precision is calculated as in eq (5):

$$Precision = \frac{\Sigma True\ Positive\ (TP)}{\Sigma\ True\ \ Positives\ (TP) + False\ Positive(FP)} \tag{5}$$

3) RECALL

The model achieved a recall score of 89%, reflecting its capability to identify most of the ambiguous requirements. A high recall is crucial in requirements engineering, as missing ambiguities can lead to misinterpretations and costly errors later in the development process. Recall measures the proportion of actual ambiguous requirements that were correctly identified by the model. While the model shows high precision, the slightly lower recall indicates that there

is still a small percentage of ambiguous requirements that were not detected. This trade-off between precision and recall is typical in classification problems and can be managed by adjusting the model parameters or using additional contextual information to improve detection rates. Recall is calculated as in eq (6):

$$Recall = \frac{\Sigma\ True\ Positives\ (TP)}{\Sigma\ True\ Positives\ (TP) + False\ Negatives(FN)} \tag{6}$$

4) F1-SCORE

The F1-score, which balances precision and recall, was found to be 92%. This high F1-score indicates that the model is well-balanced, capable of effectively identifying ambiguous requirements while minimizing both false positives and false negatives. The F1-score is particularly useful in scenarios where a balance between precision and recall is required, such as in requirements engineering, where both missed ambiguities (false negatives) and incorrect detections (false positives) can have serious implications. F1-Score is calculated as in eq (7):

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{7}$$

Model Performance on the Test Set:

- Training Set Size: 1,244 samples
- Testing Set Size: 312 samples
- TF-IDF Feature Matrix Shape (Training): (1,244, 1,000)
- TF-IDF Feature Matrix Shape (Testing): (312, 1,000)

The model was evaluated on a test set of 312 requirements, achieving the following performance metrics. Table 10 shows the overall evolution of metrics.

TABLE 10. Model evaluation metrics.

Metric	Value
Accuracy	0.97
Precision	0.97
Recall	0.89
F1-Score	0.92

Overall, the high scores across these metrics validate the efficacy of the proposed hybrid framework in detecting ambiguity within software requirements. The results indicate that the combination of rule-based and machine learning approaches can significantly improve the accuracy and reliability of ambiguity detection compared to traditional methods [7], [29]. To further analyze the model's performance, a confusion matrix was generated as shown in figure 8, providing a detailed breakdown of the model's predictions.

True Negatives (271): The model correctly identified 271 non-ambiguous requirements, demonstrating its ability to accurately distinguish clear requirements.

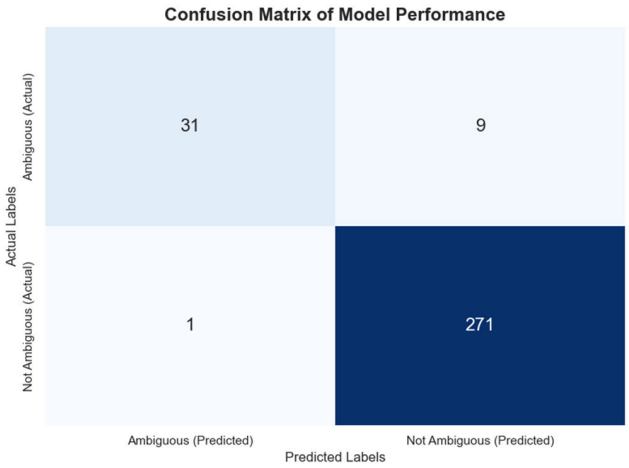


FIGURE 8. Confusion matrix of model performance.

False Positives (1): Only one non-ambiguous requirement was incorrectly classified as ambiguous, highlighting the model's high precision in ambiguity detection.

False Negatives (9): The model missed 9 ambiguous requirements, classifying them as non-ambiguous. These false negatives suggest areas where the model's recall could be improved, potentially through the incorporation of additional contextual information or refinement of the feature set.

True Positives (31): 31 ambiguous requirements were correctly identified, showcasing the model's strength in detecting genuine ambiguities.

The analysis underscores the model's high precision and good balance between precision and recall. However, the presence of false negatives indicates that there is room for improvement in capturing all types of ambiguous requirements, especially those that may not fit predefined patterns or are less common in the dataset.

IV. COMPARATIVE ANALYSIS OF MODELS

An ablation study was conducted to evaluate the proposed hybrid framework against various machine learning models (as shown in table 11) and rule-based tools (as shown in table 12) for ambiguity detection in software requirements. The study aims to analyze their effectiveness based on key performance metrics, including accuracy, precision, recall, and F1-score, alongside computational efficiency.

Figure 9 shows the confusion matrices for the Random Forest Classifier and other models highlight key insights into their performance.

The Random Forest Classifier [26] achieved the best overall performance with an accuracy of 97% and F1-score of 92%, offering a balanced trade-off between precision and recall. The Decision Tree model provided comparable results but with faster training times, making it a viable alternative for smaller datasets. Models like SVM and Logistic Regression excelled in precision but suffered from extremely low recall, indicating their limitations in detecting all

TABLE 11. Evaluation metrics for each model-machine learning.

Model	Accuracy	Precision	Recall	F1-Score	Training Time (s)	Key Observations
Hybrid Method Using Random Forest	**0.97**	**0.97**	0.89	0.92	0.35	Best overall performance with high accuracy and balanced precision and recall.
Support Vector Machine (SVM)	0.95	**1.00**	0.57	0.73	0.35	Achieved perfect precision but lower recall, missing several ambiguous requirements.
K-Nearest Neighbors (KNN)	0.88	0.64	0.23	0.33	**0.001**	Fastest training but struggled with identifying ambiguous requirements (low recall).
Multi-Layer Perceptron (MLP)	0.93	0.84	0.53	0.65	3.25	Good accuracy and precision but relatively long training time.
Logistic Regression	0.89	**1.00**	0.12	0.22	0.015	Perfect precision but extremely low recall, missing most ambiguous requirements.
Decision Tree	0.96	0.85	0.85	0.85	0.054	Balanced performance and comparable metrics to Random Forest, with faster training.
Naive Bayes	0.88	**1.00**	0.10	0.18	0.003	High precision but extremely low recall, similar to Logistic Regression.

TABLE 12. Evaluation metrics of rule based models.

Tool	Full Name	Key Indicators	Precision	Recall	Key Observations
ARM [5]	Automated Requirements Measurement	Imperatives, weak phrases, optionality, continuances, text structure, size, specification depth.	0.43	0.08	Limited recall due to a simplistic rule set. Detects ambiguity through terms like "can," "may," and weak phrases.
RCM [5]	Requirements Complexity Measurement	Vague phrases, optionality, subjectivity, conjunctions, readability index, imperatives, weak phrases.	0.68	0.38	Best precision among tools, leveraging domain-specific knowledge. Lower recall due to stricter rules.
RETA [5]	Requirements Template Analyzer	Vague terms, pronouns, passive voice, quantifiers, plural terms, complex sentences, and adverbs.	0.41	0.98	Highest recall due to extensive pattern recognition but results in many false positives. Covers a broad range of issues, including passive voice.
QuARS [5]	Quality Analyzer for Requirement Specifications	Vagueness, subjectivity, optionality, multiplicity, under specification, implicit, and weakness.	0.43	0.85	High recall in detecting issues like vagueness and optionality. Suffers from false positives and lacks domain-specific customization.

ambiguous requirements. KNN and Naive Bayes had the fastest execution times but struggled significantly with recall, making them unsuitable for ambiguity detection tasks requiring robustness.

Rule-based tools like RETA and QuARS demonstrated high recall, identifying a broad range of ambiguity indicators such as vague terms and passive voice. However, they generated numerous false positives, reducing precision. RCM performed the best in precision (0.68), leveraging domain-specific knowledge but exhibited lower recall (0.38). ARM, with its simplistic rule set, showed limited performance overall, achieving the lowest recall (0.08).

This study highlights that machine learning models, particularly Random Forest, outperform traditional rule-based tools by achieving a strong balance between precision and recall. However, tools like RCM demonstrate the importance of domain-specific knowledge in improving precision. For applications requiring exhaustive ambiguity detection, a hybrid approach that combines the adaptability of machine learning with the combination of rule-based methods can provide the most effective solution.

V. DISCUSSION, LIMITATIONS AND FUTURE WORK

This research significantly advances the automatic detection of ambiguities in software requirements by combining rule-based techniques with machine learning models, thereby contributing to the overall enhancement of software quality [48] by reducing misinterpretations and ensuring clearer communication among stakeholders. The proposed hybrid framework addresses several challenges of traditional methods, such as scalability, interpretability, and handling nuanced ambiguities in real-world requirements [2], [3]. The results demonstrate exceptional performance, achieving 97% accuracy, 97% precision, 89% recall, and a 92% F1-score. These metrics reflect a balance between precision and recall, ensuring that the model effectively identifies ambiguous requirements while minimizing false positives and negatives. By integrating rule-based interpretability with machine learning adaptability, the framework enhances both the breadth and depth of ambiguity detection.

The interpretability of the rule-based component ensures reliable identification of explicit ambiguities, such as vague terms and subjective phrases [15], while the machine learning

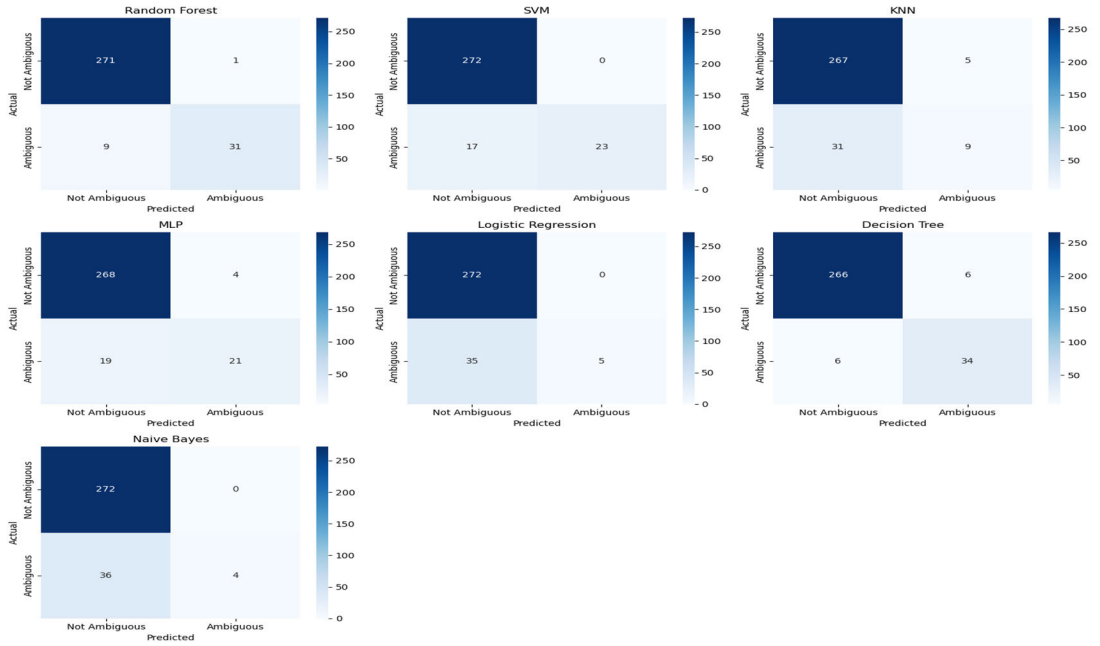


FIGURE 9. Confusion matrices comparing machine learning models for ambiguity detection in software requirements.

component excels in detecting more complex, context-sensitive ambiguities. Together, these elements create a robust solution applicable to diverse software domains. Additionally, the inclusion of sentiment analysis offers insights into stakeholder perceptions, revealing a weak positive correlation ($r = 0.22$) between ambiguity and sentiment. This suggests that some stakeholders may perceive ambiguous requirements positively due to the flexibility they offer. However, these findings also emphasize the need for balanced clarity to avoid misinterpretations while accommodating stakeholder expectations.

The clustering analysis [36] provided further insights into the linguistic patterns of requirements. It revealed that complex and verbose requirements were more prone to ambiguity, whereas concise, well-structured requirements were clearer and easier to interpret. These findings can guide practitioners in refining documentation to minimize misinterpretations. Moreover, the framework's comparative evaluation highlights its superiority over standalone rule-based tools and other machine learning models. Rule-based methods, while effective for explicit ambiguities, struggled with context-sensitive cases, whereas traditional machine learning models often exhibited trade-offs between precision and recall. The hybrid approach effectively combines their strengths, providing a comprehensive solution for ambiguity detection.

Despite its strengths, the framework has limitations that need to be addressed for further improvements. The rule-based component depends heavily on predefined patterns, which restrict its adaptability to new domains or emerging ambiguous terms. Similarly, while the machine learning component excels in identifying contextual ambiguities, it struggles with domain-specific challenges or highly pragmatic ambiguities without additional contextual data. Furthermore, the sentiment analysis employed in this study relies on basic polarity scores, which fail to capture nuanced emotional tones or context-specific sentiments [38], [39]. The dataset, while diverse, requires validation across more industries to ensure broader applicability. Lastly, the framework may face challenges in dynamic project environments where requirements evolve rapidly, requiring continuous updates to the rule set and model.

To address these limitations, several future directions are proposed. First, the integration of dynamic rule generation can enhance the framework's adaptability, allowing it to evolve continuously by learning from new data. Employing advanced NLP techniques [4], such as transformer-based models like BERT or GPT [22], could further improve the detection of nuanced ambiguities by leveraging contextual understanding. Enhancing sentiment analysis with multi-dimensional models would provide deeper insights into stakeholder perceptions and their interplay with ambiguity. Additionally, expanding validation to include more datasets from industries like education, entertainment, and human resources would test the framework's generalizability and resilience.

Developing an interactive tool that integrates real-time ambiguity detection, sentiment analysis, and clustering insights would significantly enhance the framework's practical applications. Such a tool could support requirements drafting and refinement, providing immediate feedback

to stakeholders and enabling iterative improvement of documentation. Integrating the framework into agile practices, such as sprint planning and retrospectives, could further align requirements with iterative development cycles. Context-aware clustering, which links linguistic patterns with actionable refinement strategies, could also enhance the interpretability and usability of the framework.

The hybrid framework effectively addresses ambiguity detection challenges in software requirements by integrating rule-based and machine learning approaches. The inclusion of sentiment and clustering analyses adds depth to the solution, enabling a more comprehensive understanding of requirements clarity and stakeholder perceptions. While the framework achieves exceptional performance, addressing its limitations and pursuing the proposed future directions will further enhance its robustness and applicability. These advancements will ensure the framework's continued relevance in modern software development, improving project communication, reducing ambiguity, and fostering successful outcomes.

VI. CONCLUSION

This research introduces a hybrid approach that systematically combines rule-based technique with machine learning models to detect ambiguities in software requirements. Ambiguity in requirements frequently leads to misinterpretations, implementation challenges, and projects delays. Traditional manual methods are both tedious and prone to unreliability, especially when addressing large-scale tasks. The following proposed methodology mitigates these constraints by using an initial rule-based model with a curated set of ambiguous phrases, succeeded by a machine learning model that utilizes TF-IDF vectorization and a Random Forest classifier. This dual methodology effectively tackles both simple and complex ambiguities, offering a comprehensive solution for requirements analysis. Evaluated against a dataset of 1,553 requirements from diverse project domains, the framework demonstrated significant improvements over conventional techniques in terms of accuracy, precision, recall, and F1-score. The framework integrates the interpretability of rule-based approaches with the pattern recognition skills of machine learning, guaranteeing high accuracy to minimize false positives and high recall to successfully detect ambiguous needs. Moreover, sentiment analysis yielded valuable insights into stakeholder perspectives; however, the correlation between sentiment and ambiguity remained weak. Subsequent clustering research elucidated patterns within the data, revealing distinct groupings of requirements classified by their complexity and clarity. The findings demonstrate that the hybrid technique significantly enhances ambiguity identification and fosters a deeper understanding of the requirements landscape. This research contributes to the field of automatic ambiguity detection by introducing a scalable and interpretable approach that enhances the clarity and precision of software requirements. By improving the quality of requirements documentation, this work promotes the development of more reliable and robust software systems. Furthermore, the proposed framework establishes a clear pathway for advancing natural language processing (NLP) models and adapting them to diverse datasets, thereby supporting continuous improvements in requirements engineering and effectively addressing the evolving complexities of software development.

REFERENCES

[1] T. Kato and K. Tsuda, "A method of ambiguity detection in requirement specifications by using a knowledge dictionary," *Proc. Comput. Sci.*, vol. 207, pp. 1482–1489, Jan. 2022, doi: 10.1016/j.procs.2022.09.205.

[2] D. M. Berry and E. Kamsties, "Ambiguity in requirements specification," in *Perspectives on Software Requirements*. Berlin, Germany: Springer, 2004.

[3] F. de Bruijn and H. L. Dekkers, "Ambiguity in natural language software requirements: A case study," in *Proc. 16th Int. Working Conf.* (Lecture Notes in Computer Science), vol. 6182, R. Wieringa and A. Persson, Eds., Essen, Germany. Berlin, Germany: Springer-Verlag, Jul. 2010, pp. 233–247, doi: 10.1007/978-3-642-14192-8_21.

[4] F. Fabbrini, M. Fusani, V. Gervasi, S. Gnesi, and S. Ruggieri, "Achieving quality in natural language requirements," in *Proc. 11th Int. Softw. Quality Week*, Jan. 1998.

[5] A. Bajceta, M. Leon, W. Afzal, P. Lindberg, and M. Bohlin, "Using NLP tools to detect ambiguities in system requirements—A comparison study," in *Proc. REFSQ Workshops*, 2021, pp. 1–10. [Online]. Available: https://ceur-ws.org/Vol-3122/NLP4RE-paper-3.pdf

[6] A. Fantechi, S. Gnesi, and L. Semini, "Rule-based NLP vs ChatGPT in ambiguity detection, a preliminary study," in *Proc. REFSQ Workshops*, 2023, pp. 1–10. [Online]. Available: https://ceur-ws.org/Vol-3378/NLP4RE-paper1.pdf

[7] A. Ferrari, G. Lipari, S. Gnesi, and G. O. Spagnolo, "Pragmatic ambiguity detection in natural language requirements," in *Proc. IEEE 1st Int. Workshop Artif. Intell. Requirements Eng. (AIRE)*, Karlskrona, Sweden, Aug. 2014, pp. 1–8, doi: 10.1109/AIRE.2014.6894849.

[8] S. Alharbi, "Ambiguity detection in requirements classification task using fine-tuned transformation technique," in *Machine Learning Techniques and Data Science Trends*. Chennai, India: AIRCC Publishing Corporation, 2022, pp. 1–9, doi: 10.5121/csit.2022.122105.

[9] R. S. Satpute and A. Agrawal, "A critical study of pragmatic ambiguity detection in natural language requirements," *Int. J. Intell. Syst. Appl. Eng.*, vol. 11, no. 3, pp. 249–259, 2023. [Online]. Available: https://ijisae.org/index.php/IJISAE/article/view/2681

[10] U. S. Shah and D. C. Jinwala, "Resolving ambiguities in natural language software requirements," *ACM SIGSOFT Softw. Eng. Notes*, vol. 40, no. 5, pp. 1–7, Sep. 2015, doi: 10.1145/2815021.2815032.

[11] E. Kamsties and B. Peach, "Taming ambiguity in natural language requirements," in *Proc. IEEE Joint Int. Conf. Requirements Eng.*, Dec. 2000, pp. 1–9.

[12] J. Matsuoka and Y. Lepage, "Ambiguity spotting using wordnet semantic similarity in support to recommended practice for software requirements specifications," in *Proc. 7th Int. Conf. Natural Language Process. Knowl. Eng.*, 2011, pp. 479–484.

[13] Y. Wang, I. L. M. Gutièrrez, K. Winbladh, and H. Fang, "Automatic detection of ambiguous terminology for software requirements," in *Natural Language Processing and Information Systems* (Lecture Notes in Computer Science), vol. 7934, E. Mátais, F. Meziane, M. Saraee, V. Sugumaran, and S. Vadera, Eds., Berlin, Germany: Springer, 2013, doi: 10.1007/978-3-642-38824-8_3.

[14] E. Kamsties, "Understanding ambiguity in requirements engineering," in *Engineering and Managing Software Requirements*, A. Aurum and C. Wohlin, Eds., Berlin, Germany: Springer, 2005, doi: 10.1007/3-540-28244-0_11.

[15] V. Gervasi, A. Ferrari, D. Zowghi, and P. Spoletini, "Ambiguity in requirements engineering: Towards a unifying framework," in *From Software Engineering to Formal Methods and Tools, and Back* (Lecture Notes in Computer Science), vol. 11865, M. Ter Beek, A. Fantechi, and L. Semini, Eds., Cham, Switzerland: Springer, 2019, doi: 10.1007/978-3-030-30985-5_12.

[16] A. Ferrari and A. Esuli, "An NLP approach for cross-domain ambiguity detection in requirements engineering," *Automated Softw. Eng.*, vol. 26, no. 3, pp. 559–598, Sep. 2019, doi: 10.1007/s10515-019-00261-7.
[17] F. Dalpiaz, I. V. D. Schalk, and G. Lucassen, "Pinpointing ambiguity and incompleteness in requirements engineering via information visualization and NLP," *Requirements Eng., Found. Softw. Qual.*, vol. 10753, pp. 119–135, Jan. 2018.
[18] B. Gleich, O. Creighton, and L. Kof, "Ambiguity detection: Towards a tool explaining ambiguity sources," in *Requirements Engineering: Foundation for Software Quality* (Lecture Notes in Computer Science), vol. 6182, R. Wieringa and A. Persson, Eds., Berlin, Germany: Springer, 2010, doi: 10.1007/978-3-642-14192-8_20.
[19] H. Yang, A. de Roeck, V. Gervasi, A. Willis, and B. Nuseibeh, "Analysing anaphoric ambiguity in natural language requirements," *Requirements Eng.*, vol. 16, no. 3, pp. 163–189, Sep. 2011.
[20] N. Kiyavitskaya, N. Zeni, L. Mich, and D. M. Berry, "Requirements for tools for ambiguity identification and measurement in natural language requirements specifications," *Requirements Eng.*, vol. 13, no. 3, pp. 207–239, Sep. 2008, doi: 10.1007/s00766-008-0063-7.
[21] D. M. Fernández, S. Wagner, M. Kalinowski, M. Felderer, P. Mafra, A. Vetrò, T. Conte, M. T. Christiansson, D. Greer, C. Lassenius, and T. Männistö, "Naming the pain in requirements engineering: Contemporary problems, causes, and effects in practice," *Empirical Softw. Eng.*, vol. 22, no. 5, pp. 2298–2338, Oct. 2017, doi: 10.1007/s10664-016-9451-7.
[22] K. Liu, S. Reddivari, and K. Reddivari, "Artificial intelligence in software requirements engineering: State-of-the-art," in *Proc. IEEE 23rd Int. Conf. Inf. Reuse Integr. Data Sci.*, Aug. 2022, pp. 106–111. [Online]. Available: ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9874229
[23] S.-C. Necula, F. Dumitriu, and V. Greavu-Serban, "A systematic literature review on using natural language processing in software requirements engineering," *Electronics*, vol. 13, no. 11, p. 2055, May 2024, doi: 10.3390/electronics13112055.
[24] K. Zamani, D. Zowghi, and C. Arora, "Machine learning in requirements engineering: A mapping study," in *Proc. IEEE 29th Int. Requirements Eng. Conf. Workshops (REW)*, Notre Dame, IN, USA, Sep. 2021, pp. 116–125, doi: 10.1109/REW53955.2021.00023.
[25] T. Li, X. Zhang, Y. Wang, Q. Zhou, Y. Wang, and F. Dong, "Machine learning for requirements engineering (ML4RE): A systematic literature review complemented by practitioners' voices from stack overflow," *Inf. Softw. Technol.*, vol. 172, Aug. 2024, Art. no. 107477, doi: 10.1016/j.infsof.2024.107477.
[26] N. S. Thomas and S. Kaliraj, "An improved and optimized random forest based approach to predict the software faults," *Social Netw. Comput. Sci.*, vol. 5, no. 5, p. 530, May 2024, doi: 10.1007/s42979-024-02764-x.
[27] F. Nazir, W. H. Butt, M. W. Anwar, and M. A. K. Khattak, "The applications of natural language processing (NLP) for software requirement engineering—A systematic literature review," in *Information Science and Applications* (Lecture Notes in Electrical Engineering), vol. 424, K. Kim and N. Joukov, Eds., Singapore: Springer, 2017, doi: 10.1007/978-981-10-4154-9_56.
[28] R. Izhar, K. Cosh, and S. N. Bhatti, "Enhancing agile software development: A novel approach to automated requirements prioritization," in *Proc. 21st Int. Joint Conf. Comput. Sci. Softw. Eng. (JCSSE)*, Phuket, Thailand, Jun. 2024, pp. 286–293, doi: 10.1109/jcsse61278.2024.10613648.
[29] R. Hanslo and M. Tanner, "Machine learning models to predict agile methodology adoption," in *Proc. 15th Conf. Comput. Sci. Inf. Syst.*, 2020, pp. 697–704. [Online]. Available: https://annals-csis.org/proceedings/2020/drp/214.html
[30] W. Zhang, T. Yoshida, and X. Tang, "A comparative study of TF*IDF, LSI and multi-words for text classification," *Exp. Syst. Appl.*, vol. 38, no. 3, pp. 2758–2765, 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417410008626?via=ihub
[31] G. Sidorov, F. Velasquez, E. Stamatatos, A. Gelbukh, and L. Chanona-Hernández, "Syntactic n-grams as machine learning features for natural language processing," *Exp. Syst. Appl.*, vol. 41, no. 3, pp. 853–860, 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417413006271?via%3Dihub
[32] S. Orozco-Arias, J. S. Piña, R. Tabares-Soto, L. F. Castillo-Ossa, R. Guyot, and G. Isaza, "Measuring performance metrics of machine learning algorithms for detecting and classifying transposable elements," *Processes*, vol. 8, no. 6, p. 638, May 2020, doi: 10.3390/pr8060638.
[33] S. Senbel, "Fast and memory-efficient TFIDF calculation for text analysis of large datasets," in *Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices*. Berlin, Germany: Springer, 2021, pp. 557–563, doi: 10.1007/978-3-030-79457-6_47.
[34] M. Abbas, A. Ferrari, A. Shatnawi, E. Enoiu, M. Saadatmand, and D. Sundmark, "On the relationship between similar requirements and similar software," *Requirements Eng.*, vol. 28, pp. 23–47, Jan. 2022, doi: 10.1007/s00766-021-00370-4.
[35] K. P. Sinaga and M.-S. Yang, "Unsupervised K-means clustering algorithm," *IEEE Access*, vol. 8, pp. 80716–80727, 2020.
[36] S. Naeem and A. Wumaier, "Study and implementing K-mean clustering algorithm on English text and techniques to find the optimal value of K," *Int. J. Comput. Appl.*, vol. 182, no. 31, pp. 7–14, Dec. 2018, doi: 10.5120/ijca2018918234.
[37] A. M. Ikotun, A. E. Ezugwu, L. Abualigah, B. Abuhaija, and J. Heming, "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data," *Inf. Sci.*, vol. 622, pp. 178–210, Apr. 2023, doi: 10.1016/j.ins.2022.11.139.
[38] B. Lin, F. Zampetti, G. Bavota, M. Di Penta, M. Lanza, and R. Oliveto, "Sentiment analysis for software engineering: How far can we go?" in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 94–104. [Online]. Available: ieeexplore.ieee.org/document/8453067
[39] M. Obaidi, L. Nagel, A. Specht, and J. Klünder, "Sentiment analysis tools in software engineering: A systematic mapping study," *Inf. Softw. Technol.*, vol. 151, Nov. 2022, Art. no. 107018, doi: 10.1016/j.infsof.2022.107018.
[40] C. Werner, Z. S. Li, and N. Ernst, "What can the sentiment of a software requirements specification document tell us?" in *Proc. IEEE 27th Int. Requirements Eng. Conf. Workshops (REW)*, Sep. 2019, pp. 106–107, doi: 10.1109/REW.2019.00022.
[41] V. Shukla. (2023). *Software Requirements Dataset*. [Online]. Available: www.kaggle.com/datasets/iamvaibhav100/software-requirements-dataset
[42] Supplementary Material. *ECSER: Evaluating Classifiers in Software Engineering Research*. Non Functional Requirements. Accessed: Jul. 23, 2024. [Online]. Available: https://zenodo.org/records/5336292
[43] P. Schober, C. Boer, and L. A. Schwarte, "Correlation coefficients: Appropriate use and interpretation," *Anesthesia Analgesia*, vol. 126, no. 5, pp. 1763–1768, May 2018, doi: 10.1213/ane.0000000000002864.
[44] T. S. Chu, W. Si, S. Simoff, and Q. V. Nguyen, "A machine learning classification model using random forest for detecting DDoS attacks," in *Proc. Int. Symp. Netw., Comput. Commun. (ISNCC)*, Shenzhen, China, Jul. 2022, pp. 1–7, doi: 10.1109/ISNCC55209.2022.9851797.
[45] S. Siddique, M. Naveed, A. Ali, I. Keshta, M. I. Satti, A. Irshad, Z. Alomari, O. C. Edo, and O. A. Diekola, "An effective framework to improve the managerial activities in global software development," *Nonlinear Eng.*, vol. 12, no. 1, Sep. 2023, Art. no. 20220312, doi: 10.1515/nleng-2022-0312.
[46] M. R. Dewi, I. K. Raharjana, D. Siahaan, and N. Jannah, "Extracting software requirements-related information from online news using DomText-WMDS," *Jurnal Infotel*, vol. 15, no. 3, pp. 218–226, Aug. 2023.
[47] S. Y. Puspitasari, I. A. Ra'uf, R. Nurtantyana, and C. T. Satrio, "The redevelopment of proxsis workspace with responsive design and multiplatform approaches using flutter framework," *Int. J. Inf. Commun. Technol.*, vol. 10, no. 1, pp. 41–52, Jun. 2024, doi: 10.21108/ijoict.v10i1.919.
[48] N. Nurhuda, E. Darwiyanto, and S. Widowati, "Implementation of analytical hierarchy process (AHP) for determining priority of software assessment in west Java provincial government based on ISO/IEC 25010 (case study: Sapawarga application)," *Indonesian J. Comput. (Indo-JC)*, vol. 6, no. 1, pp. 23–40, 2021, doi: 10.34818/INDOJC.2021.6.1.525.
[49] A. Aljaedi, S. Siddique, M. I. Satti, A. R. Alharbi, M. Alotaibi, and M. Usman, "Underpinning quality assurance: Identifying core testing strategies for multiple layers of Internet-of-Things-based applications," *Sustainability*, vol. 15, no. 22, p. 15683, Nov. 2023, doi: 10.3390/su152215683.
[50] A. Nadeem, M. Naveed, M. Islam Satti, H. Afzal, T. Ahmad, and K.-I. Kim, "Depression detection based on hybrid deep learning SSCL framework using self-attention mechanism: An application to social networking data," *Sensors*, vol. 22, no. 24, p. 9775, Dec. 2022, doi: 10.3390/s22249775.



RAHAT IZHAR is currently pursuing the Ph.D. degree in computer engineering with Chiang Mai University, Thailand. She has over eight years of industry experience in software requirements engineering, agile development, and ERP systems. Since August 2024, she has been a Lecturer with Krirk University, specializing in artificial intelligence and related fields. Her research interests include requirements engineering, machine learning, and agile software development, with publications in top journals and conferences.



SHAHID N. BHATTI received the Ph.D. degree in system quality engineering from Johannes Kepler University Linz, Austria, in 2007. He has more than 23 years of teaching, research, and industrial experience. He is currently a Senior Professor with the Department of Software Engineering, College of Computer Science and Engineering (CCSE), University of Jeddah, Jeddah, Saudi Arabia. His current research interests include system engineering, AI, requirements engineering, CBSD, SQA, Agile, MDA, the IoT, and reengineering.



SULTAN A. ALHARTHI received the Ph.D. degree in computer science from the Department of Computer Science, New Mexico State University, USA, in 2019. He is currently an Assistant Professor of software engineering with the University of Jeddah, Jeddah, Saudi Arabia. His current research interests include human–computer interaction, game design, cognitive-adaptive interfaces, user experience, and software engineering.